

Estudo Comparativo entre Algoritmos de Ordenação

Guilherme Andrade Nascimento Medeiros¹

¹Departamento de Computação – Universidade Federal do Espírito Santo (UFES)
Alegre – ES – Brasil

`guilherme.a.medeiros@edu.ufes.br`

Resumo. *O crescimento do volume de dados tem aumentado a demanda por métodos eficientes de organização da informação, destacando a importância dos algoritmos de ordenação na organização de estruturas lineares. Diante disso, este trabalho avalia experimentalmente o desempenho dos algoritmos de ordenação Bolha, Bolha com Critério de Parada, Inserção Direta, Inserção Binária, Inserção Ternária, Shell Sort, Seleção Direta, Heap Sort, Quick Sort, Merge Sort, Radix Sort e Bucket Sort*, analisando sua eficiência computacional por meio de três experimentos com vetores de diferentes tamanhos, cujos resultados foram comparados.*

Abstract. *The growth in data volume has increased the demand for efficient information organization methods, highlighting the importance of sorting algorithms in organizing linear data structures. In this context, this study experimentally evaluates the performance of the sorting algorithms Bubble Sort, Bubble Sort with Early Stopping Criterion, Direct Insertion Sort, Binary Insertion Sort, Ternary Insertion Sort, Shell Sort, Selection Sort, Heap Sort, Quick Sort, Merge Sort, Radix Sort, and Bucket Sort, analyzing their computational efficiency through three experiments using arrays of different sizes, whose results were subsequently compared.*

1. Introdução

Este estudo tem como objetivo avaliar, por meio de experimentação, os dados obtidos a partir da aplicação dos algoritmos de ordenação Bolha, Bolha com Critério de Parada, Inserção Direta, Inserção Binária, Inserção Ternária, Shell Sort, Seleção Direta, Heap Sort, Quick Sort, Merge Sort, Radix Sort e Bucket Sort*, bem como analisar suas respectivas eficiências e aplicabilidades. Para isso, foram realizados três testes com vetores de diferentes tamanhos para cada algoritmo, e os dados obtidos foram comparados.

Os experimentos foram iniciados com vetores de capacidade para 10.000 elementos, avaliando-se o tempo de execução, o número de comparações e o número de movimentações necessárias para a ordenação de vetores previamente ordenados em ordem crescente, em ordem decrescente e desordenados com valores aleatórios. O mesmo procedimento foi aplicado a vetores de tamanhos 100.000 e 500.000 elementos. A partir dos resultados obtidos, buscou-se identificar as vantagens e desvantagens da utilização de cada algoritmo em comparação aos demais.

O tempo de execução foi medido utilizando a função `clock()`, que retorna o tempo de CPU consumido pelo processo. Cada algoritmo foi executado uma única vez para

cada configuração de entrada, considerando-se apenas o tempo referente à ordenação e desconsiderando-se o tempo de geração dos dados.

Este trabalho está organizado da seguinte forma: na Seção 2 são apresentados, de maneira sucinta, os algoritmos de ordenação considerados neste estudo, bem como suas respectivas implementações na linguagem C; na Seção 3 são descritos os dados obtidos nos experimentos, acompanhados de uma breve análise; e, por fim, na Seção 4 são apresentadas as conclusões decorrentes da análise dos resultados.

2. Algoritmos de Ordenação

Algoritmos de ordenação são procedimentos desenvolvidos para organizar ou reorganizar um conjunto de valores dispostos em uma determinada sequência, com o objetivo de facilitar o acesso e o processamento dos dados posteriormente. Uma de suas principais aplicações é o a ordenação de vetores, pois uma única variável pode armazenar diversos elementos, conforme o tamanho definido para ele. Como exemplo prático disso, pode-se citar a organização da lista telefônica no celular, de modo que os nomes fiquem arranjados de forma alfabética.

2.1. Bolha

Trata-se de um dos algoritmos de ordenação mais simples. Seu funcionamento baseia-se na comparação de pares de elementos adjacentes, realizando a troca entre eles sempre que se encontram fora de ordem, de modo que o elemento de maior valor seja progressivamente deslocado para a direita do vetor. "O algoritmo bubble sort ordena os elementos por meio de comparações sucessivas entre posições adjacentes, repetindo esse processo até que o vetor esteja completamente ordenado", dito por [Cormen et al. 2009]. Ao final da primeira varredura completa, é possível assegurar que o maior elemento estará corretamente posicionado na última posição do vetor.

```
1 void bolha(int *vetor, int n) {  
2     int aux;  
3     for(int i = (n - 1); i >= 1; i--){  
4         for(int j = 0; j < i; j++){  
5             comparacoes++;  
6             if(vetor[j] > vetor[j+1]){  
7                 trocas++;  
8                 aux = vetor[j];  
9                 vetor[j] = vetor[j+1];  
10                vetor[j+1] = aux; } } } }
```

Fonte: Cormen (2009) (Editado pelo Autor, 2025).

2.2. Bolha com Critério de Parada

Esse algoritmo é uma variação do método tradicional de ordenação por bolha. Ele mantém o mesmo princípio de funcionamento, baseado na comparação de elementos adjacentes e na troca de posições sempre que o elemento da esquerda for maior que o da direita. De acordo com [Ziviani 2011], o bubble sort com parada antecipada melhora o desempenho da versão clássica ao interromper a execução quando nenhuma troca é realizada durante uma passagem pelo vetor, reduzindo comparações desnecessárias. Dessa forma, caso não ocorra nenhuma movimentação ao longo de uma varredura completa, é possível concluir que o vetor já se encontra ordenado.

```

1 void bolhaCP (int *vetor, int n) {
2     int aux, j;
3     int x = 1;
4     int indice = n-1;
5     while(x){
6         j = 1;
7         x = 0;
8         while(j < n){
9             comparacoes++;
10            if(vetor[j-1] > vetor[j]){
11                trocas++;
12                aux = vetor[j-1];
13                vetor[j-1] = vetor[j];
14                vetor[j] = aux;
15                x = 1; }
16            indice = j;
17            j = j+1; }
18        n = indice; } }

```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.3. Inserção Direta

O algoritmo de Inserção Direta apresenta implementação simples e possui funcionamento semelhante ao algoritmo da Bolha. Seu método baseia-se na comparação dos elementos e na inserção direta de cada valor em sua posição adequada. À medida que o algoritmo percorre o vetor, os elementos são ordenados individualmente, sendo inseridos na posição correta, de modo que cada elemento fique precedido por valores menores e sucedido por valores maiores. O algoritmo percorre o vetor da esquerda para a direita, mantendo os elementos à esquerda progressivamente ordenados [Bissoli 2025].

```

1 void insercaoDireta(int *vetor, int n) {
2     int aux, j;
3     for(int i = 1; i < n; i++){
4         aux = vetor[i];
5         j = i - 1;
6         while((j >= 0)){
7             comparacoes++;
8             if(aux < vetor[j]){
9                 vetor[j+1] = vetor[j];
10                trocas++;
11                j = j-1; }
12            else
13                break; }
14        if(j != (i-1)){
15            trocas++;
16            vetor[j+1] = aux; } } }

```

Fonte: Cormen (2009) (Editado pelo Autor, 2025).

2.4. Inserção Binária

A Inserção Binária é uma variação do algoritmo de Inserção Direta, diferenciando-se pelo método utilizado para localizar a posição correta de inserção do elemento no

vetor ordenado. Enquanto na inserção direta essa posição é encontrada por meio de comparações sequenciais, na inserção binária utiliza-se a busca binária, reduzindo o número de comparações necessárias. No entanto, apesar dessa otimização na etapa de busca, o algoritmo mantém o mesmo custo de movimentação de elementos, uma vez que os deslocamentos continuam sendo realizados de forma linear.

```
1 void insercaoBinaria(int *vetor, int n){
2     int aux;
3     int esq, dir, meio;
4     for(int i = 0; i < n; i++){
5         aux = vetor[i];
6         esq = 0;
7         dir = i;
8         while(esq < dir){
9             meio = (esq + dir) / 2;
10            comparacoes++;
11            if(vetor[meio] <= aux)
12                esq = meio + 1;
13            else
14                dir = meio; }
15        if (esq != i) {
16            for(int j = i; j > esq; j--){
17                vetor[j] = vetor[j-1];
18                trocas++; }
19            vetor[esq] = aux;
20            trocas++; } } }
```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.5. Inserção Ternária

A Inserção Ternária é outra variação do algoritmo de Inserção Direta, na qual a posição correta de inserção do elemento é determinada por meio de uma busca ternária, que divide o intervalo de busca em três partes. Essa abordagem tem como objetivo reduzir o número de comparações necessárias; entretanto, assim como nas demais variações da inserção, o custo de movimentação dos elementos permanece o mesmo, uma vez que os deslocamentos continuam sendo realizados de forma linear.

```
1 void insercaoTernaria(int *vetor, int n){
2     int aux;
3     int esq, dir, m1, m2;
4     for(int i = 0; i < n ; i++){
5         aux = vetor[i];
6         esq = 0;
7         dir = i;
8         while(esq < dir){
9             int terc = (dir - esq) / 3;
10            m1 = esq + terc;
11            m2 = dir - terc;
12            comparacoes++;
13            if(vetor[m1] > aux){
14                dir = m1;
15            }else {
16                comparacoes++;
17                if(vetor[m2] < aux)
```

```

18         esq = m2 + 1;
19     else{
20         esq = m1 + 1;
21         dir = m2; } } }
22     if (esq != i) {
23         for(int j = i; j > esq; j--){
24             vetor[j] = vetor[j-1];
25             trocas++; }
26     vetor[esq] = aux;
27     trocas++; } } }

```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.6. Shell Sort

O Shell Sort é um algoritmo de ordenação derivado do método de Inserção Direta, apresentando uma versão aprimorada desse procedimento. Ele utiliza um parâmetro de espaçamento, conhecido como gap, para reorganizar os elementos de uma estrutura por meio de inserções diretas. Esse valor define o intervalo entre os elementos que serão comparados durante o processo de ordenação. Segundo [Knuth 1998], o Shell Sort baseia-se em inserções sucessivas com espaçamentos decrescentes, sendo a sequência de incrementos por ele proposta uma das mais eficientes e amplamente estudadas. Embora essa diferença pareça sutil, sua aplicação torna o algoritmo mais eficiente, pois possibilita a reorganização simultânea de subconjuntos da estrutura, acelerando o processo de ordenação.

```

1 void shellsort(int *vetor, int n){
2     int h = 1, j;
3     int aux;
4     while(h < n / 3){
5         h = 3 * h + 1; }
6     while(h >= 1){
7         for(int i = h; i < n; i++){
8             aux = vetor[i];
9             j = i - h;
10            int move = 0;
11            while(j >= 0){
12                comparacoes++;
13                if(aux < vetor[j]){
14                    trocas++;
15                    vetor[j+h] = vetor[j];
16                    j = j - h;
17                    move = 1; }
18                else
19                    break; }
20            if (move) {
21                vetor[j+h] = aux;
22                trocas++; } }
23        h /= 3; } }

```

Fonte: Kruth (1998) (Editado pelo Autor, 2025).

2.7. Seleção Direta

O Seleção Direta é um dos algoritmos de ordenação mais simples e amplamente utilizados. Seu funcionamento consiste em selecionar, a cada etapa, o menor valor do conjunto e posicioná-lo na primeira posição disponível, seguido do segundo menor na posição seguinte, e assim sucessivamente até que todos os n elementos estejam organizados em suas respectivas posições. Ao final do processo, a estrutura fica ordenada de forma que cada elemento à esquerda apresente valor inferior ao elemento posicionado à direita.

```
1 void selecaoDireta(int *vetor, int n){
2     int menor, aux;
3     for(int i = 0; i < n - 1; i++){
4         menor = i;
5         for(int j = i + 1; j < n; j++){
6             comparacoes++;
7             if(vetor[j] < vetor[menor])
8                 menor = j; }
9         if(i != menor){
10            trocas++;
11            aux = vetor[i];
12            vetor[i] = vetor[menor];
13            vetor[menor] = aux; } } }
```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.8. Heap Sort

O Heap Sort é um algoritmo de ordenação eficiente que se baseia na estrutura de dados conhecida como heap, geralmente implementada como um heap máximo. Segundo [Bissoli 2025], um heap pode ser definido como um vetor que simula uma árvore binária completa, exceto possivelmente no último nível. Inicialmente, os elementos são organizados de modo a satisfazer a propriedade do heap, na qual o maior valor encontra-se na raiz da estrutura. Em seguida, esse maior elemento é removido e colocado na última posição do vetor, reduzindo o tamanho do heap. Após essa remoção, a estrutura é reajustada para manter a propriedade do heap. Esse processo é repetido sucessivamente até que todos os elementos estejam ordenados.

```
1 void criaHeap(int *vetor, int inicio, int final){
2     int aux = vetor[inicio];
3     int j = inicio * 2 + 1;
4     int move = 0;
5     while(j <= final){
6         if(j < final){
7             comparacoes++;
8             if(vetor[j] < vetor[j+1]){
9                 j = j + 1; } }
10        comparacoes++;
11        if(aux < vetor[j]){
12            vetor[inicio] = vetor[j];
13            trocas++;
14            inicio = j;
15            j = 2 * inicio + 1;
16            move = 1; }
17        else
```

```

18         j = final + 1; }
19     if (move) {
20         vetor[inicio] = aux;
21         trocas++; } }
22 void heapSort(int *vetor, int n){
23     int aux;
24     for(int i = (n-1)/2; i >= 0; i--){
25         criaHeap(vetor, i, n-1); }
26     for(int i = n - 1; i > 0; i--){
27         trocas++;
28         aux = vetor[0];
29         vetor[0] = vetor[i];
30         vetor[i] = aux;
31         criaHeap(vetor, 0, i - 1); } }

```

Fonte: Bissoli (2025) (Editado pelo Autor, 2025).

2.9. Quick Sort

Quick Sort por sua vez, se baseia na técnica de divisão e conquista. Inicialmente, um elemento do conjunto é escolhido como pivô, e os demais elementos são reorganizados de forma que valores menores que o pivô fiquem à sua esquerda, enquanto os maiores fiquem à sua direita. Após esse particionamento, o pivô encontra-se em sua posição correta no vetor. Em seguida, o mesmo procedimento é aplicado recursivamente aos subvetores à esquerda e à direita do pivô. Esse processo se repete até que os subvetores tenham tamanho unitário ou zero, resultando no vetor completamente ordenado. O Quick Sort analisado apresenta três variações, que evidenciam diferentes estratégias de escolha do pivô. Em uma delas, o pivô é selecionado como o elemento central do vetor; em outra, o pivô é posicionado no último elemento; e, por fim, utiliza-se a estratégia da mediana de três, considerando o menor valor, o maior valor e o elemento central do conjunto de dados.

2.9.1. Quick Sort Centro

Esta variação, ao utilizar o pivô central, favorece a formação de partições mais equilibradas, principalmente em conjuntos de dados não ordenados. Essa estratégia contribui para a redução da profundidade das chamadas recursivas, favorecendo um desempenho mais estável e próximo do caso médio do algoritmo.

```

1 int particaoCentro(int *vetor, int esq, int dir){
2     int i = esq, j = dir;
3     int aux, pivo = vetor[(esq + dir) / 2];
4     while(i <= j){
5         while(i < dir){
6             comparacoes++;
7             if(vetor[i] < pivo)
8                 i++;
9             else
10                break; }
11        while(j > esq){
12            comparacoes++;
13            if(vetor[j] > pivo)

```

```

14         j--;
15     else
16         break; }
17     if(i < j){
18         aux = vetor[i];
19         vetor[i] = vetor[j];
20         vetor[j] = aux;
21         trocas++; }
22     if(i <= j){
23         i++;
24         j--; } }
25     return i; }
26 void quickSortCentro(int *vetor, int esq, int dir){
27     if(esq < dir){
28         int i = particaoCentro(vetor, esq, dir);
29         quickSortCentro(vetor, esq, i - 1);
30         quickSortCentro(vetor, i, dir); } }

```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.9.2. Quick Sort Fim

Já esta variação, que utiliza o pivô posicionado no final do vetor, apresenta uma implementação mais simples, porém pode resultar em partições menos equilibradas, especialmente quando os dados já se encontram parcialmente ou totalmente ordenados. Nessas situações, o desempenho do algoritmo pode se aproximar do pior caso, aumentando a profundidade das chamadas recursivas e o custo computacional.

```

1 int particaoFim(int *vetor, int esq, int dir){
2     int i = esq - 1;
3     int aux, pivo = vetor[dir];
4     for(int j = esq; j < dir; j++){
5         comparacoes++;
6         if(vetor[j] <= pivo){
7             i++;
8             if(i != j){
9                 aux = vetor[i];
10                vetor[i] = vetor[j];
11                vetor[j] = aux;
12                trocas++; } } }
13     if(i + 1 != dir){
14         aux = vetor[i + 1];
15         vetor[i + 1] = vetor[dir];
16         vetor[dir] = aux;
17         trocas++; }
18     return i + 1; }
19 void quickSortFim(int *vetor, int esq, int dir){
20     while(esq < dir){
21         int i = particaoFim(vetor, esq, dir);
22         if(i - esq < dir - i){
23             quickSortFim(vetor, esq, i - 1);
24             esq = i + 1;
25         } else {
26             quickSortFim(vetor, i + 1, dir);

```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.9.3. Quick Sort Mediana

Esta última variação que utiliza a mediana de três como pivô busca selecionar, por meio de uma terceira função, um valor mais representativo do conjunto de dados, considerando o menor valor, o maior valor e o elemento central. Essa estratégia tende a gerar partições mais equilibradas, reduzindo a probabilidade de ocorrência do pior caso e proporcionando um desempenho mais consistente em diferentes cenários de entrada.

```
1 int medianaDeTres(int *vetor, int esq, int dir){
2     int meio = (esq + dir) / 2;
3     comparacoes++;
4     if(vetor[esq] > vetor[meio]){
5         int aux = vetor[esq];
6         vetor[esq] = vetor[meio];
7         vetor[meio] = aux;
8         trocas++; }
9     comparacoes++;
10    if(vetor[esq] > vetor[dir]){
11        int aux = vetor[esq];
12        vetor[esq] = vetor[dir];
13        vetor[dir] = aux;
14        trocas++; }
15    comparacoes++;
16    if(vetor[meio] > vetor[dir]){
17        int aux = vetor[meio];
18        vetor[meio] = vetor[dir];
19        vetor[dir] = aux;
20        trocas++; }
21    return vetor[meio]; }
22 int particaoMediana(int *vetor, int esq, int dir){
23     int i = esq, j = dir;
24     int aux, pivo = medianaDeTres(vetor, esq, dir);
25     while(i <= j){
26         while(i < dir){
27             comparacoes++;
28             if(vetor[i] < pivo)
29                 i++;
30             else
31                 break; }
32         while(j > esq){
33             comparacoes++;
34             if(vetor[j] > pivo)
35                 j--;
36             else
37                 break; }
38         if(i <= j){
39             if(i < j){
40                 aux = vetor[i];
41                 vetor[i] = vetor[j];
42                 vetor[j] = aux;
```

```

43         trocas++; }
44         i++;
45         j--; } }
46     return i; }
47 void quickSortMediana(int *vetor, int esq, int dir){
48     if(esq < dir){
49         int i = particaoMediana(vetor, esq, dir);
50         quickSortMediana(vetor, esq, i - 1);
51         quickSortMediana(vetor, i, dir); } }

```

Fonte: Ziviani (2011) (Editado pelo Autor, 2025).

2.10. Merge Sort

O Merge Sort é um algoritmo de ordenação cuja finalidade é reorganizar uma estrutura linear por meio do processo de divisão, ordenação e intercalação de seus n elementos. [Cormen et al. 2009] diz, "um algoritmo de ordenação baseado na estratégia de divisão e conquista, no qual o vetor é recursivamente dividido em subvetores menores que são posteriormente intercalados de forma ordenada, garantindo complexidade $O(n \log n)$ ". Ou seja, o vetor é separado em partes que são ordenadas individualmente, em seguida, as subestruturas já organizadas são reunidas, combinando seus elementos de maneira ordenada por meio da intercalação. Esse procedimento é repetido até que todos os elementos estejam reunidos em uma única estrutura totalmente ordenada.

```

1 void intercalar(int *vetor, int ini, int fim, int meio){
2     int i = ini;
3     int j = meio + 1;
4     int k = 0;
5     int aux[fim - ini + 1];
6     while(i <= meio && j <= fim){
7         comparacoes++;
8         if (vetor[i] <= vetor[j]) {
9             aux[k++] = vetor[i++];
10        } else {
11            aux[k++] = vetor[j++]; }
12        trocas++; }
13    while (i <= meio) {
14        aux[k++] = vetor[i++];
15        trocas++; }
16    while(j <= fim){
17        aux[k++] = vetor[j++];
18        trocas++; }
19    for(i = ini, k = 0; i <= fim; i++, k++){
20        vetor[i] = aux[k];
21        trocas++; } }
22 void mergeSort(int *vetor, int ini, int fim){
23     if(ini < fim){
24         int meio = (ini + fim) / 2;
25         mergeSort(vetor, ini, meio);
26         mergeSort(vetor, meio + 1, fim);
27         intercalar(vetor, ini, fim, meio); } }

```

Fonte: Cormen (2009) (Editado pelo Autor, 2025).

2.11. Radix Sort

O Radix Sort é um algoritmo de ordenação não comparativo que organiza os elementos com base nos dígitos individuais de seus valores, processando-os iterativamente do dígito menos significativo para o mais significativo. Em cada etapa, os elementos são distribuídos em estruturas auxiliares, como vetores ou buckets, de acordo com o dígito analisado, sendo posteriormente reunidos no vetor principal. Segundo [Cormen et al. 2009], o Radix Sort processa os elementos dígito a dígito, utilizando um algoritmo estável, como o Counting Sort, para realizar a ordenação em cada iteração, garantindo complexidade linear em função do número de elementos e da quantidade de dígitos. Dessa forma, o desempenho do algoritmo está diretamente relacionado ao número de passes realizados e à quantidade de redistribuições necessárias até que o conjunto esteja completamente ordenado.

```
1 void countSort(int *vetor, int n, int exp){
2     int *output = malloc(n * sizeof(int));
3     int i, count[10] = { 0 };
4     for(i = 0; i < n; i++){
5         count[(vetor[i] / exp) % 10]++; }
6     for(i = 1; i < 10; i++){
7         count[i] += count[i - 1]; }
8     for(i = n - 1; i >= 0; i--){
9         output[count[(vetor[i] / exp) % 10] - 1] = vetor[i];
10        count[(vetor[i] / exp) % 10]--;
11        trocas++; }
12    for(i = 0; i < n; i++){
13        vetor[i] = output[i];
14        trocas++; }
15    free(output); }
16 void radixSort(int *vetor, int n){
17     int max = vetor[0];
18     for(int i = 1; i < n; i++){
19         comparacoes++;
20         if(vetor[i] > max){
21             max = vetor[i]; } }
22     for(int i = 1; max / i > 0 ; i *= 10) {
23         countSort(vetor, n, i); } }
```

Fonte: Cormen (2009) (Editado pelo Autor, 2025).

2.12. Bucket Sort

O Bucket Sort não é um algoritmo baseado exclusivamente em comparações, uma vez que sua estratégia de ordenação consiste na distribuição dos elementos em baldes de acordo com seus valores. As comparações contabilizadas durante a execução do algoritmo referem-se apenas à ordenação interna de cada balde, a qual foi realizada por meio do algoritmo Insertion Sort. Neste trabalho, foram utilizados 100 baldes, cada um com capacidade máxima de 1000 elementos, implementados por meio de uma estrutura de dados (struct) para o armazenamento dos valores. Dessa forma, a maior parte do custo do algoritmo está associada às movimentações e à ordenação interna dos baldes, e não a comparações diretas entre todos os elementos do vetor principal.

```
1 #define tam_bucket 1000
```

```

2  #define num_bucket 100
3
4  typedef struct bucket{
5      int *balde;
6      int top;
7  }Bucket;
8
9  void insertion(int v[], int tam){
10     int i, j, chave;
11     for (i = 1; i < tam; i++) {
12         chave = v[i];
13         j = i - 1;
14         while (j >= 0) {
15             comparacoes++;
16             if (v[j] > chave) {
17                 v[j + 1] = v[j];
18                 trocas++;
19                 j--;
20             } else {
21                 break; } }
22         v[j + 1] = chave;
23         trocas++; } }
24 void bucketSort(int v[], int tam){
25     Bucket bucket[num_bucket];
26     int j, i, k;
27     int min = v[0], max = v[0];
28     for(i = 1; i < tam; i++){
29         comparacoes++;
30         if(v[i] < min) min = v[i];
31         comparacoes++;
32         if(v[i] > max) max = v[i]; }
33     if( max == min) return;
34     for (i = 0; i < num_bucket; i++) {
35         bucket[i].top = 0;
36         bucket[i].balde = malloc(sizeof(int) * tam); }
37     for (i = 0; i < tam; i++) {
38         int indice = (v[i] - min) * num_bucket / (max - min + 1);
39         bucket[indice].balde[bucket[indice].top++] = v[i];
40         trocas++; }
41     for(i = 0; i < num_bucket; i++){
42         if(bucket[i].top > 1)
43             insertion(bucket[i].balde, bucket[i].top); }
44     k = 0;
45     for(i = 0; i < num_bucket; i++){
46         for(j = 0; j < bucket[i].top; j++){
47             v[k++] = bucket[i].balde[j];
48             trocas++; }
49         free(bucket[i].balde); } }

```

Fonte: O Autor (2025)

3. Resultados e Discussões

A Linguagem C foi definida como meio de codificação para a realização das simulações e o Visual Studio Code como ambiente de desenvolvimento. Quanto ao hardware, a máquina em que os testes foram executados tem as seguintes configurações:

- Fabricante: Advanced Micro Devices;
- Modelo: Ryzen 5 5500;
- Processador: AMD Ryzen 5 5500;
- Frequência: 3,60 GHz;
- Memória RAM: 16,00 GB (DDR4 2400MHz);
- Sistema Operacional: Windows 11 HOME (64 bits).

3.1. Teste em um Vetor de 10.000 Elementos

A Tabela 1 apresenta os valores médios de desempenho dos algoritmos ao processarem listas previamente ordenadas em ordem crescente, decrescente e aleatória.

Tabela 1. Valores médios de desempenho para um vetor [10000]

Lista	VETOR [10000]								
	Ordem Crescente			Ordem Decrescente			Ordem Aleatória		
Algoritmo	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas
Bolha	0,0670000	49995000	0	0,1490000	49995000	49995000	0,1080000	49995000	24966738
Bolha com Crit. de Parada	0,0010000	9999	0	0,1700000	49995000	49995000	0,1320000	49992299	24782527
Inserção Direta	0,0010000	9999	0	0,0930000	49995000	50004999	0,0530000	24907718	24907719
Inserção Binária	0,0010000	113631	0	0,0720000	123617	50004999	0,0360000	119054	24883044
Inserção Ternária	0,0010000	150184	0	0,0830000	80159	50004999	0,0370000	130168	24989015
Shell Sort	0,0010000	75243	0	0,0010000	120190	93666	0,1070000	25940549	24268292
Seleção Direta	0,0660000	49995000	0	0,0680000	49995000	5000	0,0610000	49995000	9989
Heap Sort	0,0010000	244460	146954	0,0010000	226682	126694	0,0790000	17396931	9721043
Quick Sort Centro	0,0010000	139519	0	0,0010000	139518	5000	0,0010000	176058	31666
Quick Sort Fim	0,0820000	49995000	0	0,0650000	49995000	5000	0,0010000	145724	69002
Quick Sort Mediana	0,0010000	169516	0	0,0010000	169515	5002	0,0010000	192245	34512
Merge Sort	0,0010000	69008	267232	0,0010000	64608	267232	0,0010000	120384	267232
Radix Sort	0,0010000	9999	80000	0,0010000	9999	80000	0,0020000	9999	100000
Bucket Sort	0,0030000	29898	29900	0,0030000	514998	524900	0,0020000	279430	279837

Observa-se que, para o vetor previamente ordenado, os algoritmos Bolha com Critério de Parada, Inserção Direta e Radix Sort apresentaram tempos de execução e número de comparações equivalentes apresentando melhor desempenho por serem adaptativos, aproveitando o fato de o vetor já estar ordenado, o que reduz significativamente ambos os valores analisados. Entretanto, diferentemente dos dois primeiros, o Radix Sort realizou movimentações de elementos, ainda que tais operações não tenham impactado significativamente o tempo total de execução. Em contrapartida, o Quick Sort com pivô no fim apresentou o maior tempo de execução entre todos os algoritmos analisados evidenciando um desempenho inferior nesse cenário específico, pois a escolha do pivô em um vetor ordenado gera partições desbalanceadas, levando a um aumento expressivo no número de comparações e no tempo de execução, caracterizando um comportamento próximo ao pior caso.

Na ordenação de um vetor disposto em ordem decrescente, observa-se que o Radix Sort se destaca por apresentar o menor número de comparações. Embora esse algoritmo realize um número maior de movimentações, tal custo adicional não compromete seu desempenho, resultando, ainda assim, no melhor tempo de execução entre os algoritmos analisados. Em contrapartida, os algoritmos Bolha com Critério de Parada e Inserção Direta demonstraram ineficiência para o cenário avaliado, uma vez que apresentaram elevados valores de comparações, movimentações e tempo de execução.

No caso do vetor disposto em ordem aleatória, observa-se, de modo geral, que o Radix Sort apresentou a melhor relação entre número de comparações e movimentações, sendo, portanto, considerado o algoritmo de melhor desempenho nesse cenário. Em contrapartida, os algoritmos Bolha e Bolha com Critério de Parada apresentaram os piores resultados, evidenciados pelos elevados valores de comparações e trocas realizadas durante a execução.

3.2. Vetores com 100.000 e 500.000 Elementos

A seguir, os dados referentes ao vetor de tamanho 100.000 são apresentados na Tabela 2.

Tabela 2. Valores médios de desempenho para um vetor [100000]

Lista	VETOR [100000]								
	Ordem Crescente			Ordem Decrescente			Ordem Aleatória		
Algoritmo	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas
Bolha	6,8790000	4999950000	0	15,5480000	4999950000	4999950000	20,8520000	4999950000	2495317378
Bolha com Crit. de Parada	0,0010000	99999	0	18,1760000	4999950000	4999950000	21,2290000	4999792920	2500474769
Inserção Direta	0,0010000	99999	0	9,9250000	4999950000	5000049999	4,7680000	2500729929	2500729930
Inserção Binária	0,0030000	1468946	0	7,5810000	1568929	5000049999	3,5790000	1522781	2497311720
Inserção Ternária	0,0060000	1911458	0	7,6650000	1011427	5000049999	3,5640000	1681998	2491401011
Shell Sort	0,0030000	967146	0	0,0040000	1533494	1091006	0,0180000	3734962	3459187
Seleção Direta	5,8910000	4999950000	0	6,7920000	4999950000	50000	5,9390000	4999950000	99987
Heap Sort	0,0090000	3112517	1800852	0,0090000	2926640	1597432	0,0140000	3020055	1714891
Quick Sort Centro	0,0020000	1734463	0	0,0030000	1734462	50000	0,0100000	2200750	426199
Quick Sort Fim	7,7920000	4999950000	0	6,3800000	4999950000	50000	0,0090000	2116864	956752
Quick Sort Mediana	0,0030000	2034460	0	0,0030000	2034459	50002	0,0100000	2266312	454547
Merge Sort	0,0080000	853904	3337856	0,0070000	815024	3337856	0,0130000	1536551	3337856
Radix Sort	0,0150000	99999	1000000	0,0140000	99999	1000000	0,0130000	99999	1000000
Bucket Sort	0,0110000	299898	299900	0,1730000	50149998	50249900	0,0440000	25125494	25126051

De forma semelhante ao observado para o vetor de tamanho 10.000, verifica-se que, no caso de um vetor ordenado em ordem crescente, no que se refere ao número de comparações e trocas, os algoritmos Bolha com Critério de Parada e Inserção Direta mantêm-se como os de melhor desempenho, enquanto o Quick Sort com pivô no fim continua apresentando a pior eficiência. De modo análogo, para o vetor disposto em ordem decrescente, o Radix Sort apresenta o melhor desempenho, ao passo que os algoritmos Bolha com Critério de Parada e Inserção Direta demonstram menor eficiência. Já no caso do vetor em ordem aleatória, o Radix Sort também se destaca como o algoritmo mais eficiente, enquanto os algoritmos Bolha e Bolha com Critério de Parada apresentam os piores resultados.

A Tabela 3 apresenta, de forma análoga às anteriores, os dados obtidos para o vetor com 500.000 elementos.

Tabela 3. Valores médios de desempenho para um vetor [500000]

Lista	VETOR [500000]								
	Ordem Crescente			Ordem Decrescente			Ordem Aleatória		
Algoritmo	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas	Tempo(s)	Comp.	Trocas
Bolha	169,9010000	124999750000	0	387,5960000	124999750000	124999750000	539,0130000	124999750000	62533454745
Bolha com Crit. de Parada	0,0010000	499999	0	468,2300000	124999750000	124999750000	565,5120000	124999172725	62443237460
Inserção Direta	0,0010000	499999	0	248,6430000	124999750000	125000249999	119,3100000	62432103406	62432103378
Inserção Binária	0,0180000	8475732	0	190,1600000	8975713	125000249999	91,1760000	8774139	62421828884
Inserção Ternária	0,0320000	10848582	0	189,8750000	5734280	125000249999	90,4450000	9929069	62530399958
Shell Sort	0,0130000	5601426	0	0,0250000	8622570	6197117	0,1070000	25940549	24268292
Seleção Direta	149,7050000	124999750000	0	161,1400000	124999750000	250000	148,2820000	124999750000	499980
Heap Sort	0,0480000	17837785	10105698	0,0480000	16977997	9168447	0,0790000	17396931	9721043
Quick Sort Centro	0,0150000	9737855	0	0,0140000	9737854	250000	0,0490000	12329462	2656139
Quick Sort Fim	197,7870000	124999750000	0	160,1380000	124999750000	250000	0,0510000	14203227	4560062
Quick Sort Mediana	0,0190000	11237852	0	0,0190000	11237851	250002	0,0520000	12666013	2755433
Merge Sort	0,0420000	4783216	18951424	0,0400000	4692496	18951424	0,0740000	8837733	18951424
Radix Sort	0,0890000	499999	6000000	0,0850000	499999	6000000	0,0720000	499999	5000000
Bucket Sort	0,0110000	1499898	1499900	4,3370000	1250749998	1251249900	1,0640000	624634203	624634718

Mais uma vez, observa-se a repetição do padrão identificado anteriormente. Para o vetor previamente ordenado, os algoritmos Bolha com Critério de Parada e Inserção Direta apresentaram o melhor desempenho, enquanto o Quick Sort com pivô no fim manteve resultados insatisfatórios. Ademais, os resultados obtidos para os vetores dispostos em ordem decrescente e aleatória mostraram-se consistentes com aqueles observados em experimentos realizados com vetores de menor dimensão.

4. Conclusão

O algoritmo Bolha, embora apresente implementação simples, não demonstra desempenho satisfatório, especialmente no que se refere ao elevado número de comparações realizadas. Essa limitação implica um alto consumo de recursos computacionais, o que pode resultar em lentidão e longos tempos de execução em sistemas com capacidade de processamento restrita. Sua variação, o Bolha com Critério de Parada, por outro lado, apresenta bom desempenho quando aplicada a vetores previamente ordenados; entretanto, nos demais cenários, tende a exibir desempenho semelhante ou até inferior ao do algoritmo Bolha tradicional. Dessa forma, a aplicação desses algoritmos em projetos práticos é, em geral, caracterizada como ineficiente ou de baixo desempenho.

O algoritmo Inserção Direta é amplamente empregado na ordenação de estruturas lineares de pequeno porte ou em listas parcialmente ordenadas. A Inserção Binária e a Inserção Ternária são variações da Inserção Direta, diferenciando-se pelo método utilizado para localizar a posição de inserção do elemento, por meio de busca binária e ternária, respectivamente. Embora reduzam o número de comparações, essas variações não diminuem o número de movimentações, o que limita seus ganhos de desempenho em relação à Inserção Direta. Assim como o Bolha com Critério de Parada, a Inserção Direta apresenta excelente desempenho quando aplicada a vetores previamente ordenados. Contudo, os elevados valores de comparações e movimentações apresentados pelo algoritmo nos demais cenários tornam seu desempenho insatisfatório.

O Shell Sort apresentou, de modo geral, valores de comparações e movimentações inferiores aos da maioria dos algoritmos analisados, refletindo um desempenho relativamente eficiente nos diferentes cenários avaliados. Esse comportamento deve-se à sua estratégia de ordenar elementos distantes entre si por meio de incrementos, o que reduz significativamente o número de movimentações quando comparado à Inserção Direta. Tal abordagem permite que o vetor se torne parcialmente ordenado antes das etapas finais, resultando em maior eficiência prática, especialmente em vetores de tamanho médio. Entretanto, seu desempenho não é considerado excepcional, pois depende fortemente da sequência de incrementos adotada.

O algoritmo Seleção Direta apresenta desempenho insatisfatório em todos os cenários analisados, em razão do elevado número de comparações realizadas, o que o posiciona entre os algoritmos com maior tempo de execução. Esse comportamento é explicado por sua complexidade assintótica de $O(n^2)$, uma vez que, independentemente da ordem inicial dos dados, o algoritmo percorre repetidamente todo o vetor para selecionar o menor elemento, não se beneficiando de vetores parcialmente ou totalmente ordenados.

O Heap Sort apresenta tempo de execução relativamente rápido e consistente tanto para vetores dispostos em ordem crescente quanto decrescente. Contudo, o custo associado ao número de comparações e movimentações permanece elevado quando comparado

a outros algoritmos analisados. Esse comportamento decorre da necessidade de manter a propriedade do heap ao longo de todo o processo de ordenação, o que exige sucessivas operações de ajuste da estrutura, resultando em um número elevado de comparações e movimentações, ainda que a complexidade assintótica do algoritmo permaneça $O(n \log n)$ em todos os casos.

O algoritmo Quick Sort, ao utilizar a escolha de um pivô e a subsequente subdivisão do vetor, contribui significativamente para a redução do tempo de execução médio do algoritmo. Contudo, o número de comparações e movimentações ainda pode ser elevado, dependendo da estratégia adotada para a escolha do pivô. Em particular, na variação que seleciona o pivô no final do vetor, o desempenho torna-se inferior quando aplicada a vetores previamente ordenados em ordem crescente ou decrescente. Nesses casos, ocorrem partições desbalanceadas, o que provoca um aumento expressivo no número de comparações e, conseqüentemente, uma elevação significativa no tempo de execução.

O Merge Sort apresenta elevada velocidade de execução; contudo, ainda realiza um número significativo de comparações e movimentações durante o processo de ordenação. Esse comportamento decorre de sua estratégia de divisão e conquista, na qual o vetor é recursivamente subdividido e posteriormente intercalado. Embora essa abordagem garanta complexidade $O(n \log n)$ em todos os casos, ela exige o uso de estruturas auxiliares, o que aumenta o número de movimentações e o consumo de memória durante a execução.

O Bucket Sort, ao distribuir os elementos em baldes definidos por intervalos e ordená-los separadamente, contribui para a redução do tempo de execução. Entretanto, sua principal desvantagem reside no elevado número de comparações e movimentações realizadas, o que pode resultar em maior consumo de recursos computacionais. Dessa forma, em máquinas com menor capacidade de processamento, a execução desse algoritmo pode ser prejudicada, impactando negativamente seu desempenho.

O Radix Sort, baseado nos dados deste trabalho, apresentou os resultados mais satisfatórios, especialmente quando aplicado a estruturas de grande dimensão e elevado grau de desordem. Por ser um algoritmo de ordenação não comparativo, reduz a dependência de comparações diretas entre elementos, o que contribui para melhor desempenho e menor impacto sobre os recursos computacionais.

Referências

Bissoli, D. C. (2025). Notas de aula.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. The MIT Press, 3 edition.

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition.

Ziviani, N. (2011). *Projeto de Algoritmos: com Implementações em Pascal e C*. Cengage Learning, 2 edition.